

Online Quiz Application - Project Documentation

Project ID: 65HIBKJS

Intern Name: Ujwal Hiranwar

1. Introduction

Project Overview and Purpose

This document provides a detailed overview of the Online Quiz Application, a full-stack web application developed as part of the InnoByte Services internship project. The application is designed to be a robust platform for creating, managing, and taking quizzes online. It features distinct roles for regular users and administrators, a secure authentication system, and real-time feedback on quiz performance. The application is also containerized using docker for ease of deployment.

Key Features and Functionality

- **User Authentication:** Secure user registration and login system using JWT (JSON Web Tokens).
- **Role-Based Access Control:** Separate dashboards and permissions for USER and ADMIN roles.
- **Quiz Management (Admin):** Admins can create, update, and delete quizzes, manage quiz topics, difficulty levels, and time limits.
- **Question Management (Admin):** Admins can add, edit, and delete questions (both single and multiple choice) within each quiz.
- **Quiz Taking (User):** Users can browse available quizzes, take them within a timed environment, and submit their answers. He can also get feedback on every question just after submitting the question and He will get final score after finishing the test.
- **Scoring and Results:** Automatic scoring upon quiz completion with detailed results, including score, percentage, and correct/incorrect answers.
- **Quiz History:** Users can view their past quiz attempts and track their performance over time.
- **Leaderboards:** Global and quiz-specific leaderboards to foster a competitive environment.

Target Audience

- **Students/Users:** Individuals looking to test their knowledge on various subjects.
- **Quiz Administrators/Educators:** Users responsible for creating and managing the quiz content.

Technologies Used

- **Backend:**
 - Java 17
 - Spring Boot 3.x
 - Spring Security (with JWT)
 - Spring Data JPA (Hibernate)
 - MySQL 8.x
 - Maven
 - **Frontend:**
 - React.js
 - Tailwind CSS
 - Axios (for API communication)
 - **Deployment:**
 - Docker
 - Vercel (Frontend)
-

2. System Requirements

Hardware Requirements

- **Processor:** 2 GHz Dual-Core (or equivalent) minimum
- **RAM:** 4 GB minimum, 8 GB recommended
- **Storage:** 10 GB of free space

Software Requirements

- **Operating System:** Windows, macOS, or a modern Linux distribution.
 - **Java Development Kit (JDK):** Version 17 or higher.
 - **Build Tool:** Apache Maven 3.6+
 - **Database:** MySQL Server 8.x
 - **Containerization:** Docker (for container-based deployment)
 - **Web Browser:** A modern web browser like Google Chrome, Firefox, or Edge.
 - **IDE (Optional):** IntelliJ IDEA or VS Code with Java extensions.
-

3. Installation & Setup Instructions

There are two ways to set up and run the application: using Docker (recommended for ease of use) or building from the source code.

Option 1: Install and Run using Docker

This is the simplest way to get the application running.

1. **Prerequisites:** Ensure you have Docker installed and running on your system.
2. **Run the Container:** Execute the following command in your terminal. This command will pull the image from Docker Hub and start the application.

Docker command : `docker run -d -p 8081:8081 -e DB_URL="your_database_url" -e DB_USERNAME="your_database_username" -e DB_PASSWORD="your_database_password" ujwalhiranwar/online-quiz-application:v2`

You have to add your own database credentials in this command.

3. **Access the Application:** The backend will be running and accessible at <http://localhost:8081>.
4. **Access the frontend by clicking below link and you are ready to use it.**

Frontend Application link : <https://online-quizz-frontend.vercel.app/>

Option 2: Install and Run from Source (Git Clone)

This method is for developers who want to run the application from the source code.

1. **Prerequisites:** Ensure you have JDK 17, Maven, and MySQL, Git installed.
2. **Clone the Repository:**
Git clone command: `git clone https://github.com/Ujwal-Hiranwar/online-quiz-backend.git`
3. **Database Setup:**
 - Make sure your MySQL server is running.
 - Create the database and a dedicated user by executing the following SQL Queries `CREATE DATABASE quiz_application;`
4. **Configure the Backend:**
 - Navigate to `Backend/application/src/main/resources/application.properties`.
 - Update the `spring.datasource.url`, `spring.datasource.username`, and `spring.datasource.password` properties to match your MySQL setup.
 - Add URL `"jdbc:mysql://localhost:3306/quiz_application"` name and add your other mysql credentials like `USERNAME` and `PASSWORD`
5. **Run the Backend:**
 - Open a terminal in the `Backend/application` directory.
 - Run the following Maven command `mvn spring-boot:run`
 - The backend will start on port 8081.
6. **Run the Frontend:**
 - The frontend will be accessible at <https://online-quizz-frontend.vercel.app/> and will connect to the backend.

4. Database Schema

The application uses a relational database schema to store all its data. The core entities are:

- **User:** Stores user account information, including credentials and roles.
- **Quiz:** Stores information about each quiz, such as title, topic, and difficulty.
- **Question:** Represents a single question within a quiz.
- **QuestionOption:** Stores the possible answers for a multiple-choice question.
- **QuizAttempt:** Records a user's attempt at a particular quiz.
- **UserAnswer:** Stores the specific option(s) a user selected for a question during an attempt.

Table Structures

Users :

Column	Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for the user
username	VARCHAR(50)	NOT NULL, UNIQUE	User's chosen username
email	VARCHAR(100)	NOT NULL, UNIQUE	User's email address
password	VARCHAR(255)	NOT NULL	Hashed password
first_name	VARCHAR(50)		User's first name
last_name	VARCHAR(50)		User's last name
role	VARCHAR(20)	NOT NULL	USER or ADMIN
active	BOOLEAN	NOT NULL	Whether the user account is active

quizzes

Column	Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for the quiz
title	VARCHAR(200)	NOT NULL	Title of the quiz
description	TEXT		Detailed description
topic	VARCHAR(100)	NOT NULL	Subject or topic of the quiz
difficulty_level	VARCHAR(255)		EASY , MEDIUM , HARD
time_limit_minutes	INT		Time limit in minutes
created_by	BIGINT	FOREIGN KEY (users.id)	The admin who created the quiz

questions :

Column	Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for the question
quiz_id	BIGINT	FOREIGN KEY (quizzes.id)	The quiz this question belongs to
question_text	TEXT	NOT NULL	The text of the question
question_type	VARCHAR(255)	NOT NULL	SINGLE_CHOICE , MULTIPLE_CHOICE
points	INT	NOT NULL	Points awarded for a correct answer

question_options

Column	Type	Constraints	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for the option
question_id	BIGINT	FOREIGN KEY (questions.id)	The question this option belongs to
option_text	TEXT	NOT NULL	The text of the answer option
is_correct	BOOLEAN	NOT NULL	Whether this is a correct option

5. User Guide

For Regular Users

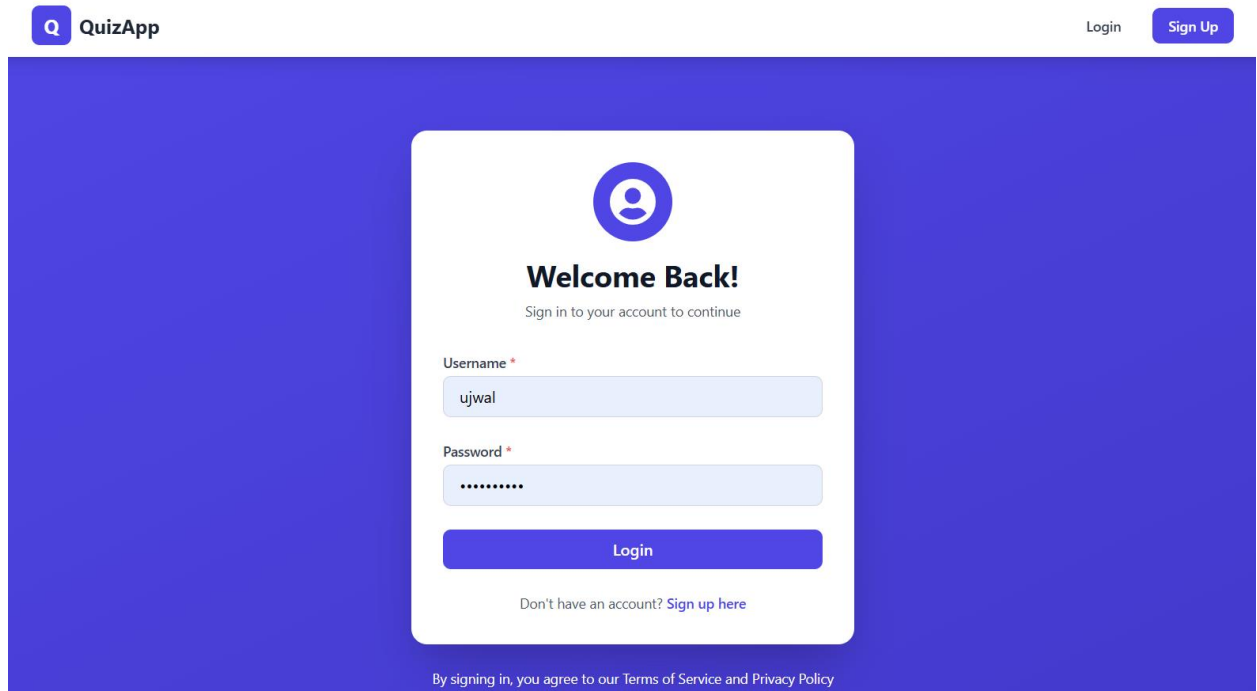
1. Registration:

- Navigate to the “Sign Up” page.
- Fill in your details (username, email, password).
- Click “Sign Up” to create your account.

The screenshot shows the 'Create Account' page of the QuizApp. The page has a green background. At the top left is the QuizApp logo, and at the top right are 'Login' and 'Sign Up' links. The main content is a white card with a green header containing a user icon and the text 'Create Account' and 'Join us and start your quiz journey'. Below this are five input fields: 'Full Name *' (placeholder: 'Enter your full name'), 'Username *' (placeholder: 'ujwal'), 'Email Address *' (placeholder: 'Enter your email'), 'Password *' (placeholder: '*****'), and 'Confirm Password *' (placeholder: 'Confirm your password'). A blue 'Sign Up' button is at the bottom of the card. Below the button is a link: 'Already have an account? Sign in here'.

2. Login:

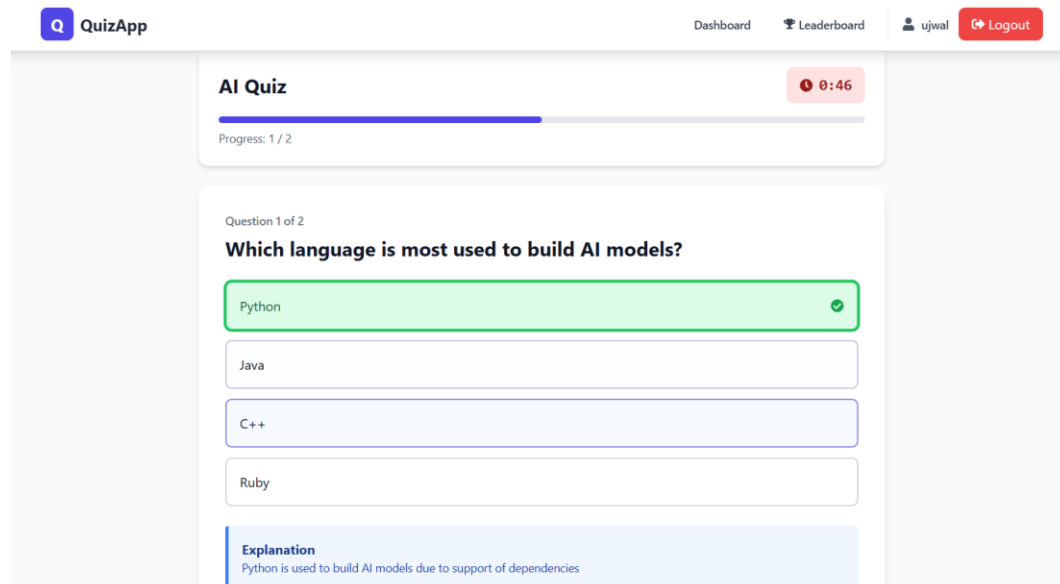
- Navigate to the “Login” page.
- Enter your credentials and click “Login”.



The image shows the login page of QuizApp. The page has a dark blue background. At the top left is the QuizApp logo, and at the top right are links for "Login" and "Sign Up". In the center, there is a white card with a user icon and the text "Welcome Back!". Below this, it says "Sign in to your account to continue". There are two input fields: "Username" with the value "ujwal" and "Password" with masked characters. A blue "Login" button is below the password field. At the bottom of the card, it says "Don't have an account? Sign up here". At the very bottom of the page, a small text line reads "By signing in, you agree to our Terms of Service and Privacy Policy".

3. Taking a Quiz:

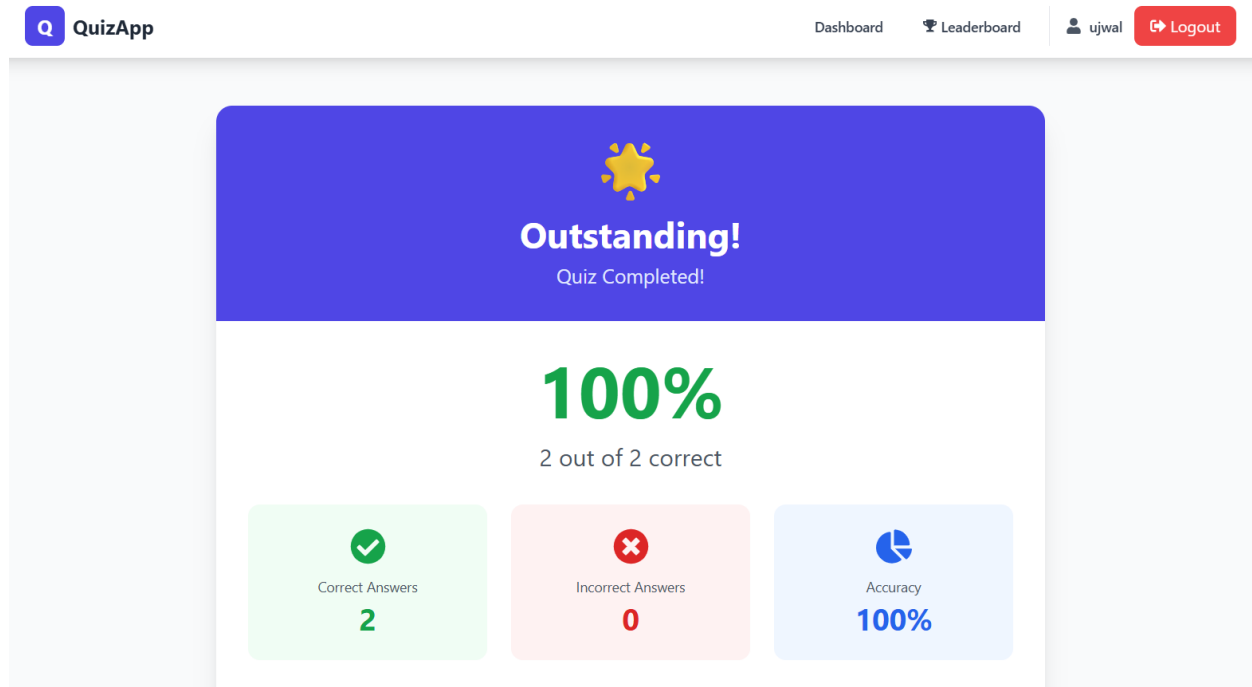
- From your dashboard, browse the list of available quizzes.
- Click “Start Quiz” on the quiz you wish to take.
- Answer each question. The timer will be visible if the quiz is timed.
- Submit the quiz once you are finished.



The image shows the quiz interface of QuizApp. At the top, there is a navigation bar with the QuizApp logo, "Dashboard", "Leaderboard", and a user profile "ujwal" with a "Logout" button. The main content area is titled "AI Quiz" and shows a progress bar at "Progress: 1 / 2". A timer in the top right corner indicates "0:46". The question is "Which language is most used to build AI models?". There are four radio button options: "Python" (selected), "Java", "C++", and "Ruby". Below the options, there is an "Explanation" section that states "Python is used to build AI models due to support of dependencies".

4. Viewing Results:

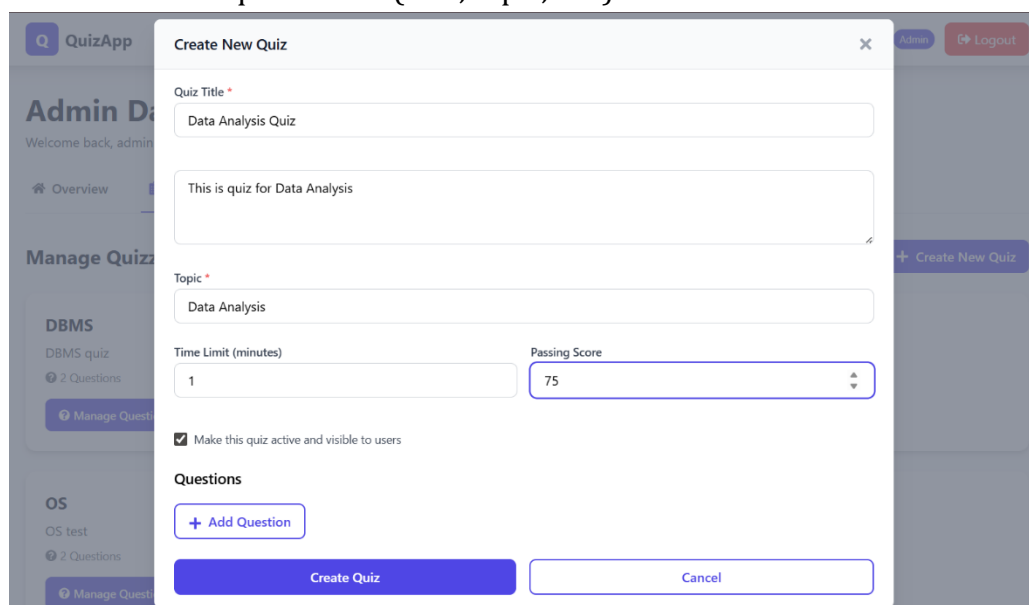
- After submitting a quiz, you will be redirected to the results page.
- Here you can see your score, percentage, and a summary of correct/incorrect answers.



For Administrators

1. Creating a Quiz:

- From the admin dashboard, navigate to the “Manage Quizzes” tab.
- Click “Create New Quiz”.
- Fill in the quiz details (title, topic, etc.) and save.



The screenshot shows the 'Create New Quiz' form in the QuizApp admin dashboard. The form includes fields for 'Quiz Title' (filled with 'Data Analysis Quiz'), a description area (filled with 'This is quiz for Data Analysis'), 'Topic' (filled with 'Data Analysis'), 'Time Limit (minutes)' (filled with '1'), and 'Passing Score' (set to '75'). There is a checkbox labeled 'Make this quiz active and visible to users' which is checked. Below these fields is a 'Questions' section with an '+ Add Question' button. At the bottom are 'Create Quiz' and 'Cancel' buttons. The background shows the admin dashboard with a sidebar containing 'Overview', 'Manage Quizzes', and 'DBMS' sections.

2. Managing Questions:

- After creating a quiz, click “Manage Questions”.
- On the question management page, click “Add Question”.
- Fill in the question text, options, select the correct answer(s), and save.
- You can edit or delete existing questions from this page.

*

The screenshot shows the 'Manage Questions' interface for a quiz titled 'AI Quiz'. The page header includes the QuizApp logo, navigation links for 'Dashboard' and 'Leaderboard', and user information for 'admin' with an 'Admin' role and a 'Logout' button. The quiz title 'AI Quiz' is displayed with a subtitle 'This is AI Quiz' and a '+ Add Question' button. A progress indicator shows '2 Questions' with a red progress bar. The first question, 'Q1 Multiple Choice', asks 'Which language is most used to build AI models?'. The options are Python (selected with a green checkmark), Java, C++, and Ruby. An explanation box states: 'Explanation: Python is used to build AI models due to support of dependencies'. At the bottom, there are 'Edit' and 'Delete' buttons.

3. Leaderboard feature : users can see how better they have performed In the leaderboard.

The screenshot shows the 'Quiz Leaderboard' page. The header includes the QuizApp logo, navigation links for 'Dashboard' and 'Leaderboard', and user information for 'disha' with a 'Logout' button. A 'Back to Dashboard' button is located at the top left. The main heading is 'Quiz Leaderboard'. Below it, a dropdown menu labeled 'Select a Quiz:' shows 'AI Quiz' selected. A table displays the leaderboard data:

RANK	PLAYER	CORRECT ANSWERS	ACCURACY
#1	ujwal	2/2	100.00%
#2	disha	1/2	50.00%

6. Architecture & Design

The backend follows a classic **layered architecture**, which promotes separation of concerns and makes the application easier to maintain and scale.

- **Controller Layer (/controller):** Exposes the REST API endpoints. It handles incoming HTTP requests, validates them, and delegates the business logic to the Service Layer.
- **Service Layer (/service):** Contains the core business logic of the application. It orchestrates calls to the Repository Layer and implements the main functionalities like creating quizzes, calculating scores, etc.
- **Repository Layer (/repository):** Responsible for all data access. It uses Spring Data JPA to interact with the database, providing a clean abstraction over database operations.
- **Entity Layer (/entity):** Defines the JPA entities that are mapped to the database tables.
- **DTO Layer (/dto):** Data Transfer Objects are used to transfer data between the client and the server, preventing direct exposure of JPA entities.
- **Security Layer (/security):** Handles authentication and authorization using Spring Security and JWT.

This design follows the **Model-View-Controller (MVC)** pattern, where Spring Boot acts as the Controller and Model, and the React frontend serves as the View.

7. Code Documentation

Package Structure

The backend source code is organized into the following packages under `com.quiz.application`:

- `config`: Spring configuration classes (e.g., `SecurityConfig`, `JpaConfig`).
- `controller`: REST controllers for handling API requests.
- `dto`: Data Transfer Objects for API request/response bodies.
- `entity`: JPA entity classes representing database tables.
- `exception`: Custom exception classes and a global exception handler.
- `repository`: Spring Data JPA repository interfaces.
- `security`: JWT-based security components (`JwtTokenProvider`, `UserDetailsService`, etc.).
- `service`: Service classes containing the application's business logic.

Key Classes

- **`QuizController.java`**: Handles all HTTP requests related to quizzes (create, fetch, delete).

- **QuizService.java**: Implements the business logic for quiz management.
 - **Quiz.java**: The JPA entity representing a quiz.
 - **AuthController.java**: Manages user registration and login endpoints.
 - **AuthService.java**: Contains the logic for user authentication and token generation.
 - **JwtAuthenticationFilter.java**: A filter that intercepts incoming requests to validate the JWT token and set up the Spring Security context.
-

8. Security Implementation

Security is a critical component of this application.

- **Authentication**: The application uses **JWT (JSON Web Tokens)** for stateless authentication. When a user logs in, a signed JWT is generated and sent to the client. This token must be included in the Authorization header of all subsequent requests to protected endpoints.
 - **Authorization**: Spring Security is used to enforce authorization rules. Endpoints are protected based on user roles (USER, ADMIN) using method-level security (`@PreAuthorize`) and request matchers in `SecurityConfig`.
 - **Password Security**: User passwords are never stored in plain text. They are securely hashed using the **BCrypt hashing algorithm** before being saved to the database.
 - **Data Validation**: Input from clients is validated at the DTO level using `jakarta.validation` annotations (e.g., `@NotBlank`, `@Email`) to prevent invalid or malicious data from entering the system.
-

9. Features Implemented

- **User Features:**
 - User Registration and Login
 - View All Active Quizzes
 - Search and Filter Quizzes
 - Take Quiz with Timer
 - View Instant Quiz Results
 - View Quiz History
 - View Quiz-Specific Leaderboards

Q QuizApp

DashboardLeaderboardujwalLogout

Welcome back, ujwal!

Ready to test your knowledge today?

OverviewAvailable Quizzes

Quizzes Taken
4

Average Score
62.5%

Best Score
100%

Total Points
5

Quick Actions

Browse QuizzesView HistoryLeaderboard

- **Admin Features:**
 - Admin Dashboard with Application Statistics
 - Full CRUD (Create, Read, Update, Delete) for Quizzes
 - Full CRUD for Questions within a Quiz

Q QuizApp

DashboardLeaderboardadminAdminLogout

Admin Dashboard

Welcome back, admin!

OverviewManage QuizzesManage Users

Total Quizzes
4

Total Users
4

Quick Actions

Manage QuizzesQuiz ResultsManage Users

10. Assumptions & Limitations

Assumptions

- It is assumed that the user has a stable internet connection.

- The application is designed for modern web browsers; compatibility with older browsers is not guaranteed.
- The user provides valid and correct information during registration.

Limitations

- The application does not currently support different question types beyond single and multiple choice (e.g., fill-in-the-blanks, true/false).
- The user interface is functional but basic; it is not designed for extensive customization.
- There is no functionality for users to reset their passwords via email.
- Not integration of AI (Artificial Intelligence) so that admin can add question based on topic and number of questions within few seconds.

Potential Future Improvements

- Implement password reset functionality.
- Add support for more question types.
- Add integration of AI to add questions automatically based on topic.
- Enhance the admin dashboard with more detailed analytics and charts.
- Allow users to create and share their own quizzes.
- Add pagination to quiz and leaderboard lists to improve performance with large datasets.